

Robust Distributed Training

TrainVerify ('25) and Oobleck ('23)

Presented by Kevin, Minkyu, Matt, and Maddy
February 3rd 2026

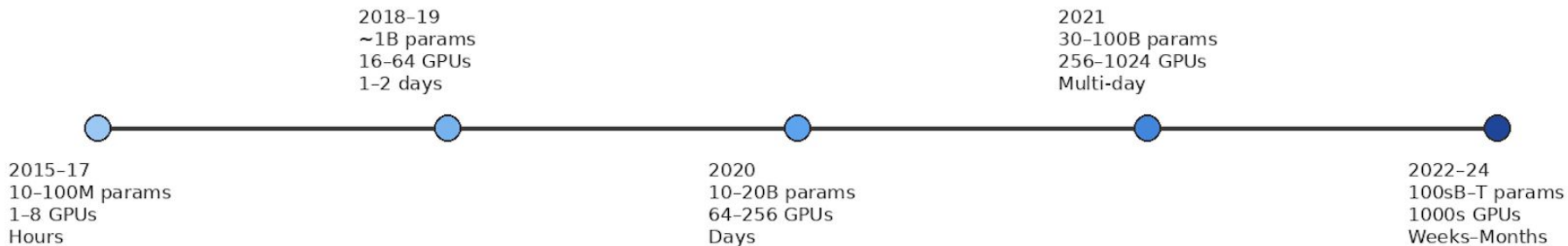
Agenda

1. Introduction
2. TrainVerify
3. Oobleck
4. SuperBench (Optional Reading)
5. Conclusion + Thoughts
6. Q&A

Introduction

Pattern 1: LLM Pretraining Gets More Expensive

The Rapid Growth of LLM Pretraining



Orders-of-magnitude growth in parameters, compute, time, and cost

 Model	 Training Scale	 Time	 Cost
Llama3	up to 16K H100 GPUs	~ 3 months	~ \$100 M
DeepSeek-V3	2K H100 GPUs	~ 2 months	~ \$6 M
PaLM	6K v4 TPUs	~ 3 months	~ \$20 M

Losing even a small fraction of training progress is no longer acceptable

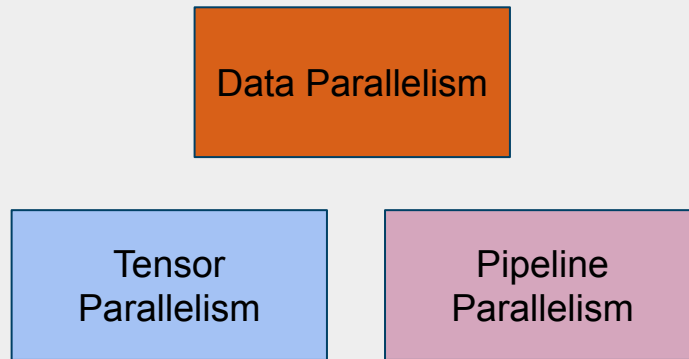
Pattern 2: Training Pipelines Grow More Complex and Rigid

Past



- Minimal sharding
- Single stage
- Simple communication

Today



- Sharding on many layers
- Dozens of stages
- Complex state and communication

Failures invalidate larger portions of work. Correctness becomes harder to reason about

Reliability Becomes Paramount

Software bugs become:

- More common
- Harder to detect
- More expensive to deal with

Hardware failures become:

- More common
- Longer to recover from
- Much bigger blast radius

Can we mitigate or prevent the effects of these problems?

TrainVerify

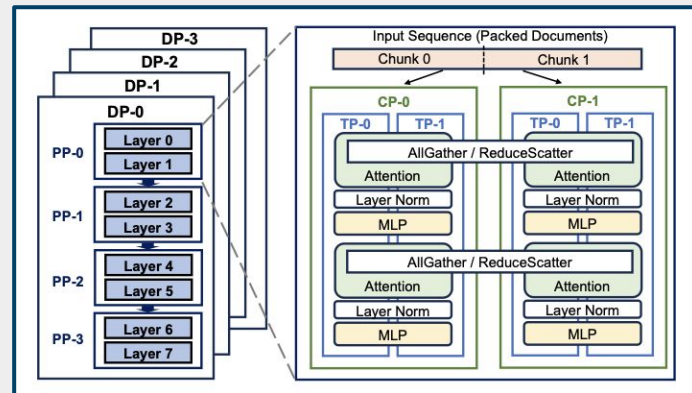
Oobleck

TrainVerify (Lu et al., 2025)

Parallelism is Hard

- Modern pipelines use **1,000-10,000+ GPUs**
- Scheduling and coordination
 - **Spatial-Temporal**
- **Silent bugs waste resources**
 - GPUs are expensive
 - Difficult to find

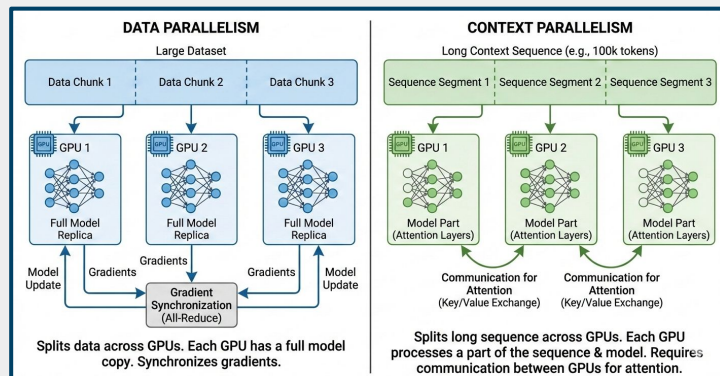
Cause	MegatronLM	DeepSpeed	nnScaler
op-transformation	16	18	19
scheduling	4	1	4
communication	8	13	5
Total	26	28	25



Loss Scaling in Megatron

- Megatron uses **tensor, data and context parallelism**
- Tensor parallelism does not affect loss semantics
- DP ranks calculate loss locally and average globally
- Two ways to scale CP loss
 - Loss centric
 - Gradient centric (Megatron)
 - Calculate local loss
 - Average gradients across CP ranks

$$L_{global} = \sum_{i=1}^C L_i$$



Megatron Does Both?

- Gradient-Centric Loss (Megatron)
 - Each CP rank calculates loss locally
 - Loss is all-reduced across the CP group
 - **Reduced loss is multiplied by context_parallel_size**
 - Gradients are divided by (dp x cp)
 - Gradients are all-reduced across (dp + cp) group
- Gradient is now affected by CP size
 - Gradient sharing expects local CP losses
- Results in noisier updates

Megatron Gradient

$$\nabla_{\theta} L_{scaled} = \sum_{i=1}^C \nabla_{\theta} L_i$$

Correct Gradient

$$\nabla_{\theta} L_{scaled} = \frac{1}{C} \sum_{i=1}^C \nabla_{\theta} L_i$$

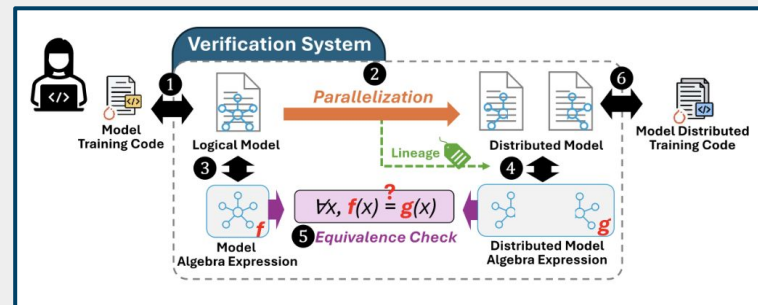
Other Silent Bugs

- **Inconsistent tensor partitioning**
 - Gradients are incorrectly computed from non-matching slices of data
- **Incorrect computation operator parallelism**
 - Changing locally computed partial results can change operator semantics
- **Missing or wrong communication operator**
 - Results may be under/over-counted
- **Incorrect communication group**
 - Ranks are separated/combined improperly

```
def training_code_gpu3():  
    model_tp_group = dist.new_group(ranks=[2, 3])  
    1 ... # upstream model code  
    gy3 = ...  
    w3 = ...  
    gx3 = gy3.matmul(w3) 2  
    3 dist.all_reduce(gx3, group=model_tp_group) 4  
    ... # downstream model code
```

TrainVerify

- Establish correctness *before* training
- Compare and verify **execution plans**
 - Establish **parallelization equivalence**
 - Mathematically prove **execution plan is equivalent to logical plan**
- Eliminates major classes of bugs
 - Does not guarantee elimination of *all* bugs



Parallelization Equivalence (PE)

- TrainVerify proves correctness by comparing **execution plans**
 - **Definition 1:** “A parallel computation is correct if, for any input, its outputs match the sequential outputs after combining parallel results”
- Checks PE by building SMT (Satisfiable Modulo Theories) formulas and applying a solver
 - SMT Problem extends the SAT Problem (NP-Hard)

G_p is parallelization equivalent to G_l iff:

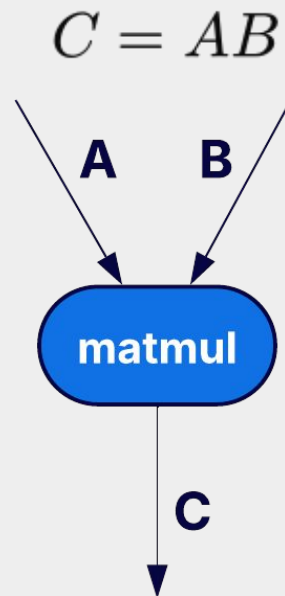
$$\forall t \in E_t, \forall x \in \chi, t(x) = \oplus_{t \in M(t)} t(x)$$

$t(x)$ denotes the concrete value of a tensor t given a concrete input x

$\oplus$ is a composition operation determined by the parallelization scheme

Obtaining Execution Plans - DFGs

- TrainVerify proves correctness of **execution plans** by **comparing Logical Definitions**
 - Checking every operator scales poorly
 - **Represented as a Data Flow Graph (DFG)**
 - **Nodes:** Represent operators
 - **Edges:** Destination operator consumes resultant tensor from the source

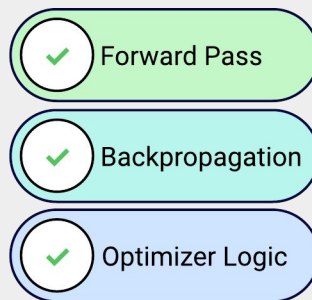


Symbolic DFGs (sDFGs)

- **TrainVerify extends DFGs** to include critical information left out by traditional frameworks
 - E.g. backpropagation, optimizer logic, metrics, etc.
- **DFGs are also symbolized**
 - Operations are explicitly replaced with algebraic representations
 - Numeric types are replaced with symbolic real types to avoid precision errors
 - See “Defeating Nondeterminism in LLM Inference”, Thinking Machines

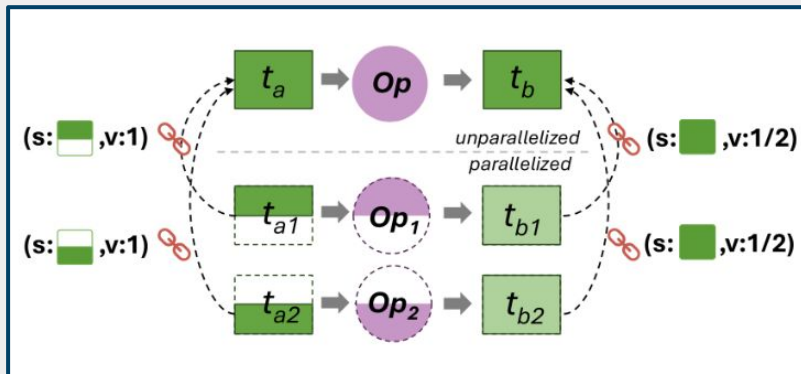
$$C = AB$$

$$C_{i,j} = \sum_k A_{i,k} \cdot B_{k,j}$$



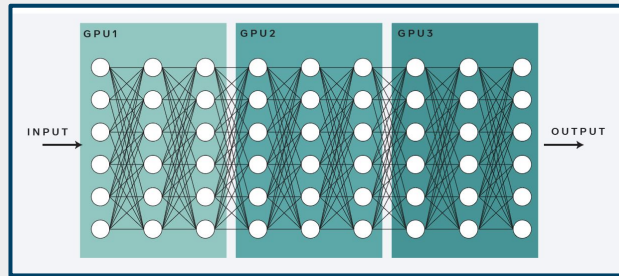
Lineage

- Data structure for **how a tensor is partitioned/replicated during parallelization**
 - Implies how to recover the logical tensor
- **Critical bridge between logical and parallelized sDFGs**
- Not all tensors have lineage
 - Tensors created during parallelization



Staged Verification

- Directly applying the solver to the generated formulas is too expensive
 - The SMT problem is NP-Hard
- **Staged Verification:** split the execution plan into stages that can be checked independently
 - **If all stages can be verified, then the execution plan can be verified**
 - **Uses lineage** to determine which tensors belong to which stages
 - Determines stage boundaries via **backward slicing**



Parallel Solving

- **Stage outputs are assumed correct** when a worker starts
 - Allows a **greater degree of parallelism**
- **Inputs are checked** against verified global relation pool
- **Output is verified** when a worker finishes
- **User can bypass certain restrictions**
 - ResNet, Deepseek

Algorithm 1 Stage Parallel Verification

Input: Logical graph G_l , multi-device graph G_p , lineage L
Output: A Boolean indicating whether G_l is equivalent to G_p .

```
1:  $R \leftarrow$  equivalence of  $G_l$  and  $G_p$  inputs  $\triangleright$  global relation pool
2: stages  $\leftarrow$  align_and_partition( $G_l, G_p, L$ )
3: for stage  $S$  in stages do
4:   async run WORKER( $S, R$ .snapshot())
5:    $R$ .add( $S$ . $L_{out}$ )
6: return all( $[S$ .worker.result() $==$ True for  $S$  in stages])
7:
8: function WORKER( $S, R$ )
9:   if ( $S$ . $L_{in}$  not in  $R$ ) then
10:    return False
11:    $X \leftarrow$  infer_rx_shapes( $S$ . $G_l, S$ . $G_p, S$ . $L_{in} \cup S$ . $L_{out}$ )
12:    $S$ .init_symbolic_variables(shape= $X$ )
13:   outputs  $\leftarrow$  symbolically execute  $S$ . $G_l$  and  $S$ . $G_p$ 
14:   cond  $\leftarrow S$ . $G_l$ .inputs  $\stackrel{S.L_{in}}{==} S$ . $G_p$ .inputs
15:   target  $\leftarrow S$ . $G_l$ .outputs  $\stackrel{S.L_{out}}{==} S$ . $G_p$ .outputs
16:   if (target is not AlwaysTrue given outputs  $\cup$  cond) then
17:     return False
18:   return True
```

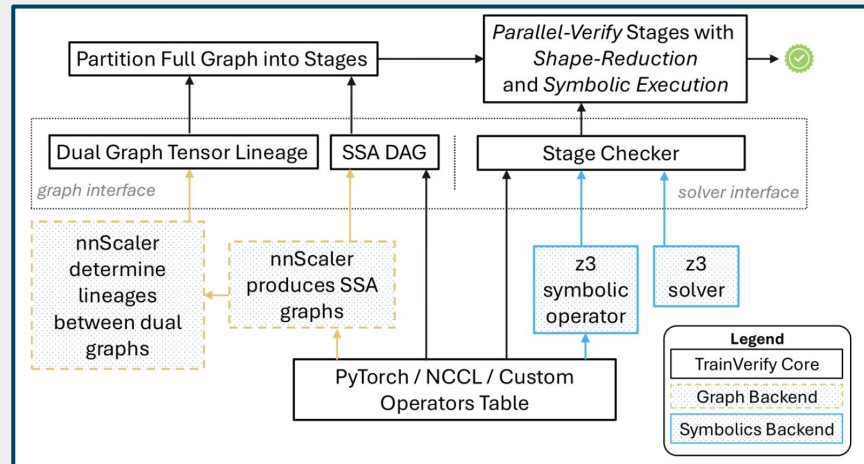
Shape Reduction

- Even with Parallel Solving, proofs are still too expensive
 - **Reduce workload by shrinking dimensions of tensor shapes**
- **SIMD instructions** perform the same operation on multiple variables
 - **Only need to verify one**
- Reduced shapes must adhere to multiple constraints
 - **Shape alignment**
 - **Semantic intactness**
 - **Lineage consistency**

$$\begin{aligned}
 c_{1,1} &= a_{1,1} \cdot b_{1,1} + a_{1,2} \cdot b_{2,1} + a_{1,3} \cdot b_{3,1} \\
 c_{2,2} &= a_{2,1} \cdot b_{1,2} + a_{2,2} \cdot b_{2,2} + a_{2,3} \cdot b_{3,2}
 \end{aligned}$$

TrainVerify Summary

1. Compute DFGs
2. Apply DFG transformations and compute lineage
3. Separate execution plans into stages
4. Check for Parallel Equivalence through an SMT solver in parallel



TrainVerify Evaluation

Verifying Real-World LLM Parallelization

TrainVerify completes verification within a **half-hour** for moderate-scale plans and a **half-day** for large-scale plans.

	Llama3			DeepSeek-V3		
	L1	L2	L3	D1	D2	D3
Solver Parallelism	30	30	30	30	30	10
End-to-end Time	0.2h	2.4h	8.0h	0.4h	2.4h	9.0h

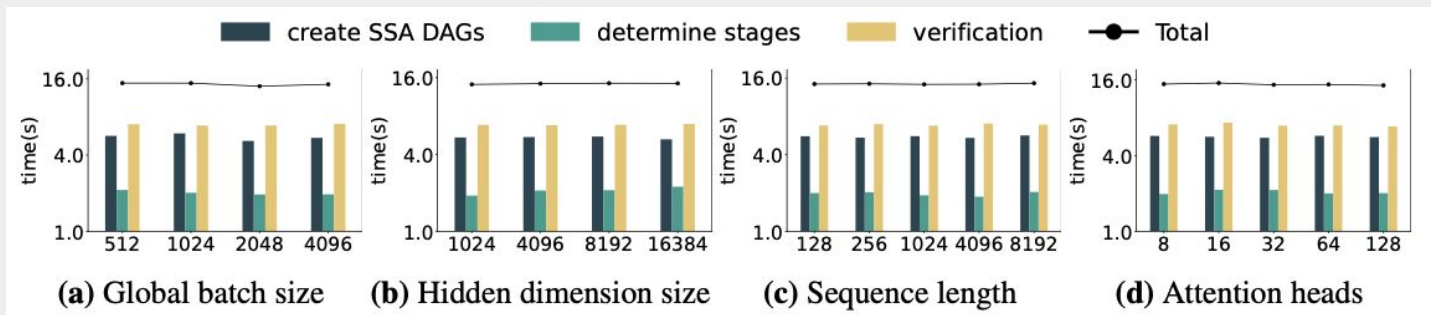
Table 3. Verification time for the evaluated models.

Exp. ID	Model	Layers	DP	TP	PP	NM
L1	Llama3-8B	32	512	1	1	1
L2	Llama3-70B	80	16	8	4	32
L3	Llama3-405B	126	64	8	16	16
D1	DS-V3-16B	27	16	4	2	16
D2	DS-V3-236B	60	16	8	4	16
D3	DS-V3-671B	61	32	8	8	16

Table 2. Evaluated real-world large models.

Invariance to Original Tensor Shapes

Verification time is **independent** of actual tensor sizes due to shape reduction.





Verification Time

Main Process Time

(mostly sequential)

- Iterates over stages
- Inspects tensor shard metadata
- Prepares solver inputs



Parallel Solver Time

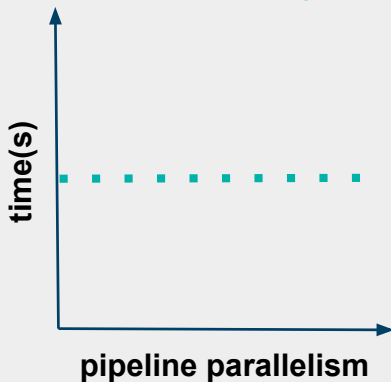
(parallel across stages)

- Initializes symbolic variables
- Performs equivalence checking
- Runs per stage (in parallel)

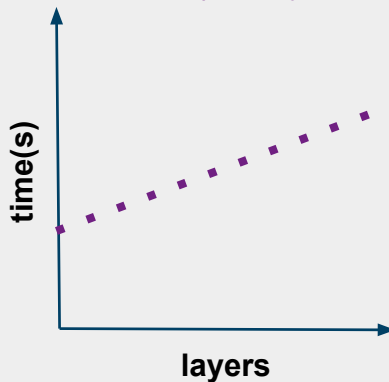
Verification time is split between lightweight orchestration and heavyweight parallel solving.

Linearity of Verification Time for Parallelization

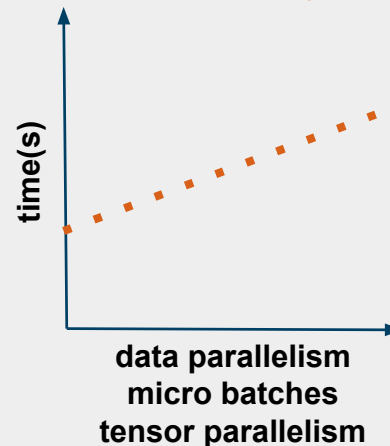
sub-linear increase due to small increase in comm. ops for each additional stage



linear increase in stages and time, as adding model layers adds operators sequentially



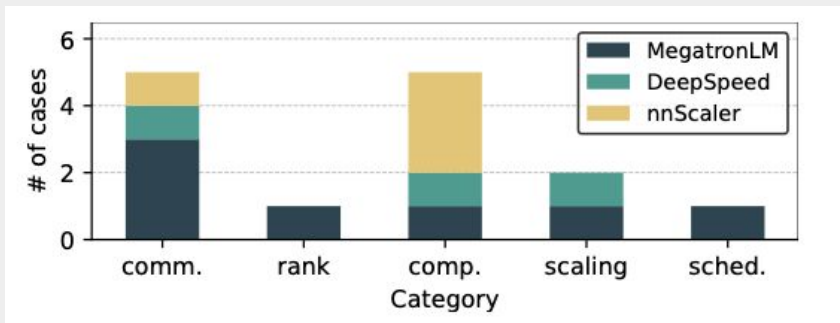
linear increase due to copied tensors, along with the introduction of communication operators



As the degree of parallelism increases, parallel solver time gradually dominates verification time, leading to quadratic (or greater) complexity in the solver.



Eliminating Bugs



TrainVerify is able to verify incorrect parallelization for these reduced cases:

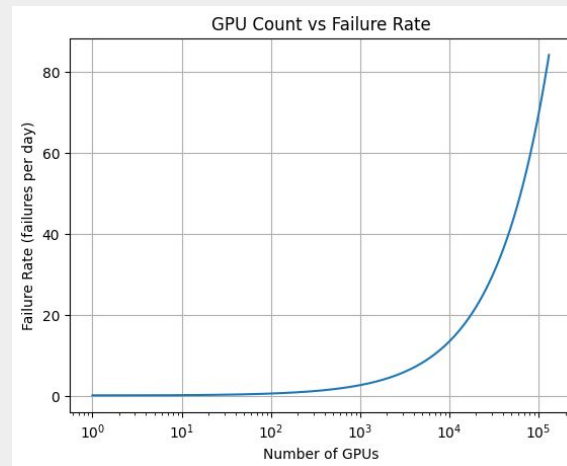
1. **comm**: incorrect communication operators
2. **rank**: incorrect device assignment
3. **comp**: incorrect partitioning of computational operators
4. **scaling**: incorrect scaling of distributed states
5. **sched**: incorrect pipeline scheduling

Bugs that live below the graph abstraction, including local buffer updates and incorrect buffer management, are out of scope.

Oobleck (Jang et al., 2023)

Hardware Failures are Inevitable

- Training runs last weeks to months
- Jobs span **hundreds to thousands** of GPUs
- Hardware failures, i.e. thermal shutdown, now occur on the order of hours or even minutes
- Each failure is more expensive



Source: Nebius, Fault-tolerant training: How we build reliable clusters for distributed AI workloads (Aug. 28, 2025).
nebius.com/blog/posts/how-we-build-reliable-clusters.

Previous Techniques

Checkpointing and Restart (**Varuna**)

- Periodically save model state
- On failure:
 - reload checkpoint
 - **restart** execution

Redundant Computation (**Bamboo**)

- Avoid **restarting**
- Each stage has at least two replicas assigned to it
- When a failure occurs, replicas bootstrap a new node

Restart overhead dominates as failure rate increases:

- Checkpointing preferred when failure rate is low, redundant computation when it is high

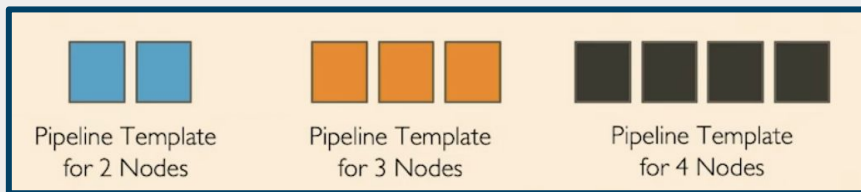
Both approaches trade useful work for reliability, and the cost grows with scale!

+

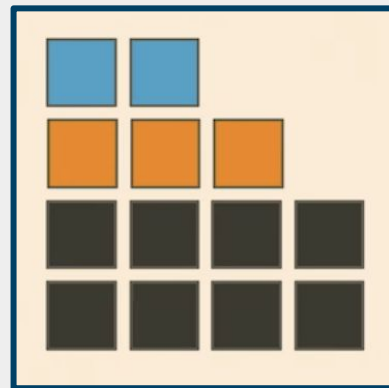
Neither have strong guarantees for number of permissible faults

Oobleck

- Configure execution structure before training begins (templates)
 - Reuse execution structure when instantiating or updating a pipeline
 - Amortize coordination across failures
 - Ensures fault tolerance of f nodes

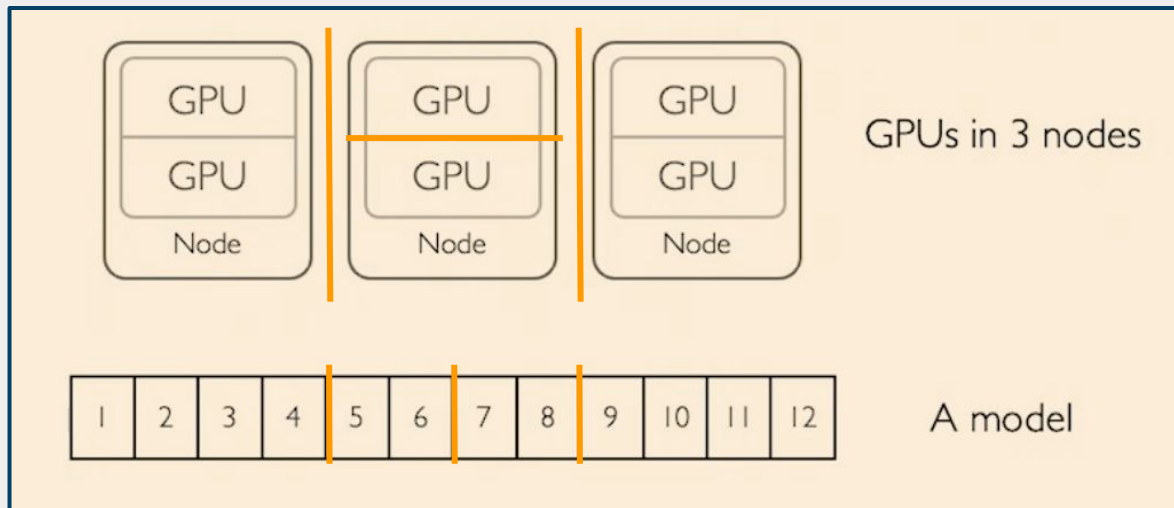


Big Idea: Separate planning and execution!



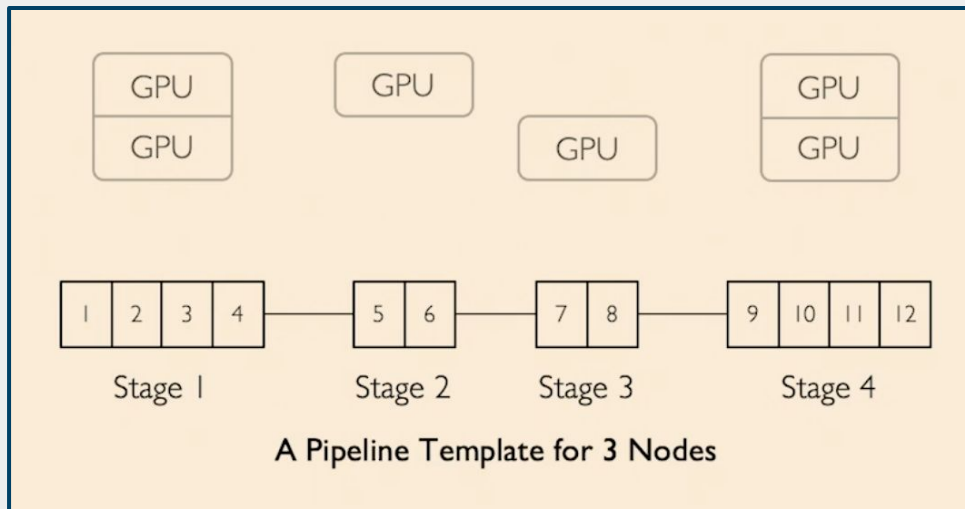
Templates

- Pre-generated execution structure description for a pipeline



Templates

- Each template is logically equivalent to one another
 - Note: Oobleck primarily deals with pipeline parallelism



Oobleck Design

1. Generate Pipeline Template
2. Instantiate Pipeline Replicas from Templates & Distribute training work across replicas
3. Detects Node Failure/Addition & Recovery

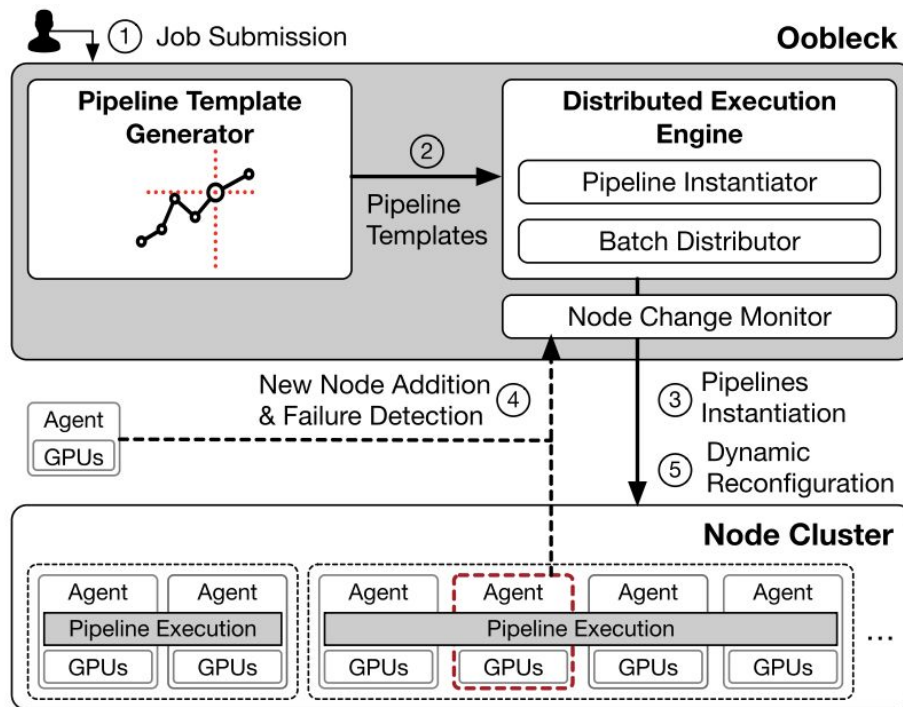


Figure 3. *Oobleck system overview.*

1. Generate Pipeline Template

2. Instantiate Pipeline Replicas from Templates & Distribute training work across replicas

3. Detects Node Failure/Addition & Recovery

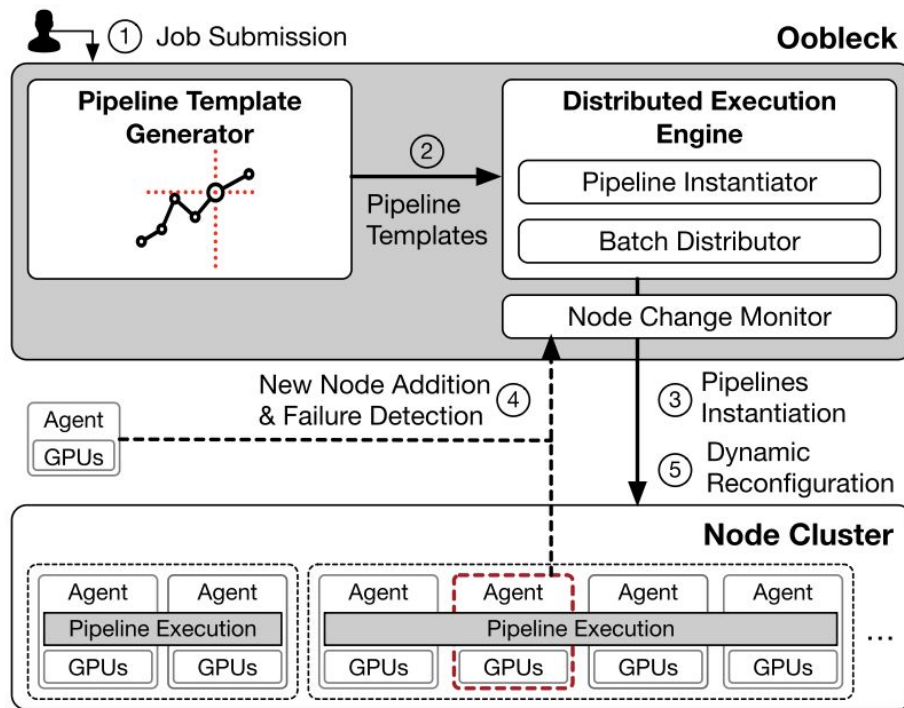
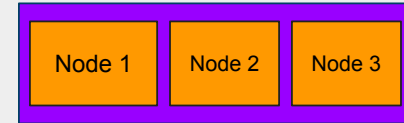


Figure 3. *Oobleck system overview.*

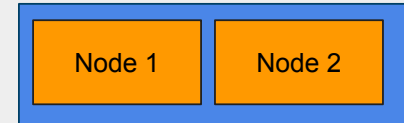


GOAL: Ensure some combination of templates can always maximize hardware utilization

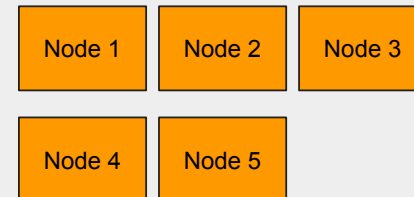
- Figure out how many templates are needed
 - Generating few restricts the flexibility
 - Generating too much is expensive
- How many nodes each template should use
 - Use Frobenius Problem



Template 1



Template 2

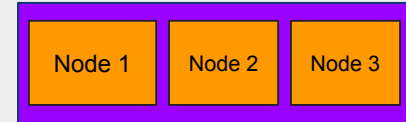


$N = 5$

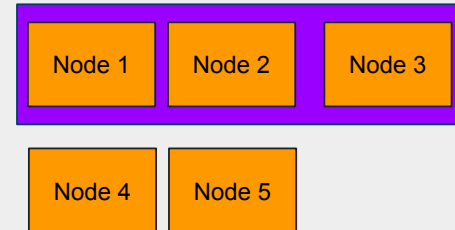


GOAL: Ensure some combination of templates can always utilize any number of available nodes

- Figure out how many templates are needed
 - Generating few restricts the flexibility
 - Generating too much is expensive
- How many nodes each template should use
 - Use Frobenius Problem



Template 1

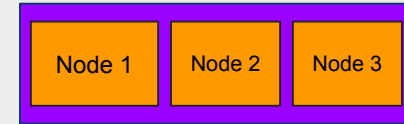


$$N = 5$$

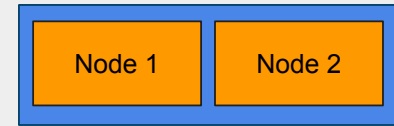


GOAL: Ensure some combination of templates can always utilize any number of available nodes

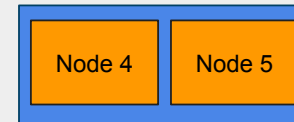
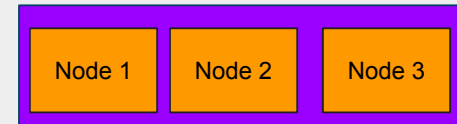
- Figure out how many templates are needed
 - Generating few restricts the flexibility
 - Generating too much is expensive
- How many nodes each template should use
 - Use Frobenius Problem



Template 1



Template 2



$N = 5$

How many templates & How many nodes for each template?

Frobenius Problem

Frobenius Problem guarantees that a linear combination of the set of pipeline templates use all nodes even after failure

Frobenius Problem (Example)

- Generate the template that uses smallest amount of nodes (n_0)
- Increment the number of nodes for next templates
- Make a template until the number of nodes reaches $N - f * n_0$
 - N = number of nodes
 - f = maximum allowable number of failures

If $n_0 = 2$, $N = 13$, $f = 4$ then make a template until the number of node reaches $13 - 4 * 2 = 5$

Template 1
of Node: 2

Template 2
of Node: 3

Template 3
of Node: 4

Template 4
of Node: 5

1. Generate Pipeline Template
2. **Instantiate Pipeline Replicas from Templates & Distribute training work across replicas**
3. Detects Node Failure/Addition & Recovery

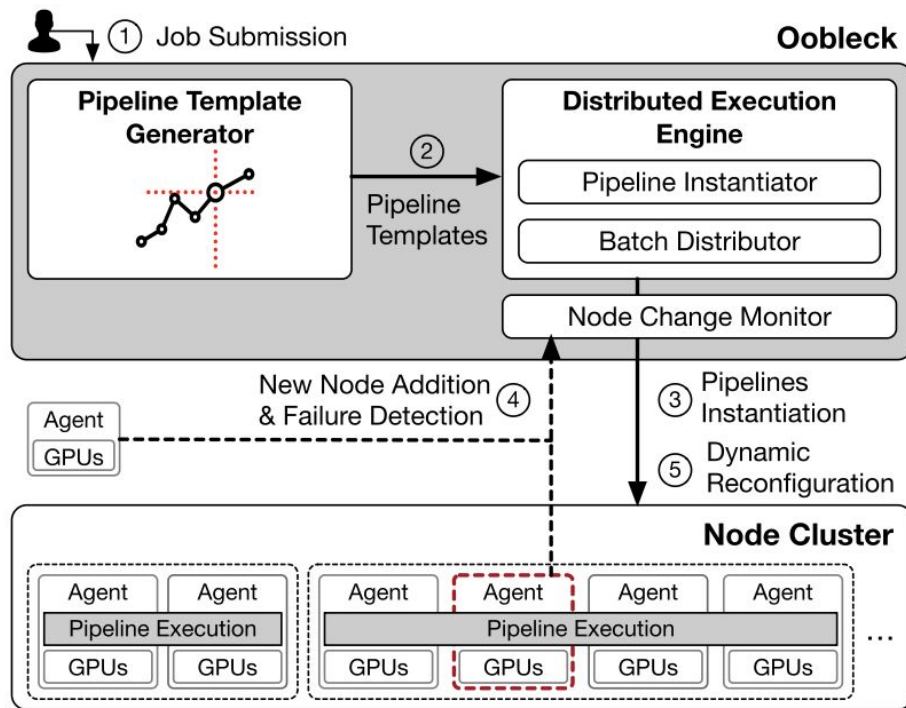


Figure 3. *Oobleck system overview.*

Optimal Pipeline & Batch Distribution Combination

Multiple Pipeline combination exists

ex) Total 14 Nodes

$$= 2 * (5 \text{ node pipeline}) + 1 * (4 \text{ node pipeline})$$

$$= 4 * (2 \text{ node pipeline}) + 2 * (3 \text{ node pipeline})$$

1. Uses Dynamic Programming to enumerate feasible pipeline combinations
2. Distribute batches across pipelines so that all pipelines complete their work in approximately same amount of time.
3. Choose the combination with the highest estimated throughput

Plan 1

Minibatch	# Instantiations	Pipeline Templates
79	x1	
119	x1	
157	x2	

Plan 2

N/A	x0	
118	x3	
158	x1	

Plan 3

79	x5	
117	x1	
N/A	x0	

1. Generate Pipeline Template
2. Instantiate Pipeline Replicas from Templates & Distribute training work across replicas
3. **Detects Node Failure/Addition & Recovery**

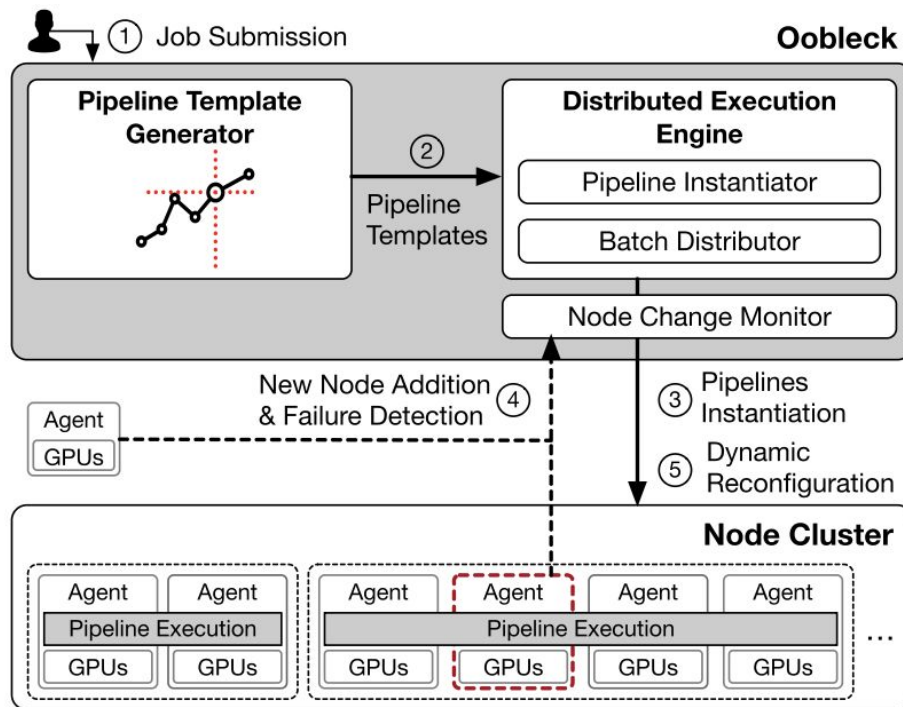
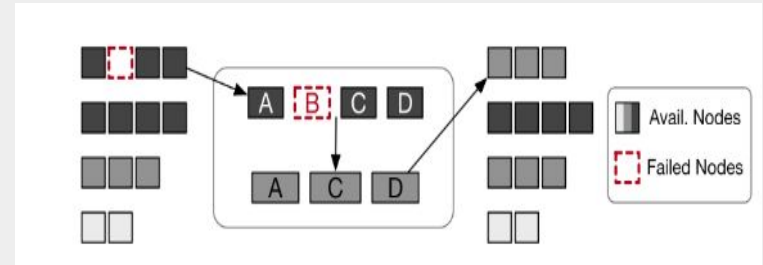


Figure 3. *Oobleck system overview.*

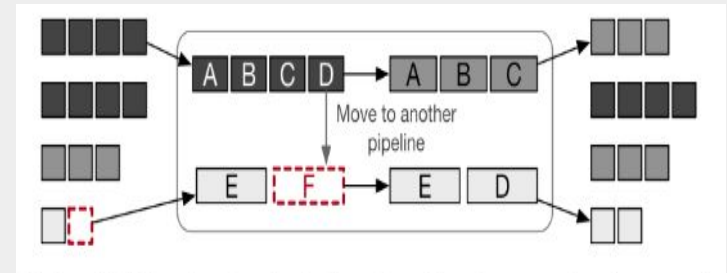
Recovery Mechanism

- IF there **EXISTS** a template that can accommodate remaining nodes, instantiate using the template
- IF there **ISN'T** a template that can accommodate remaining node,
 - If there exists a node that can be borrowed from other pipeline and reinstantiate two pipelines
 - If nodes can't be borrowed, merge with other pipeline and reinstantiate.



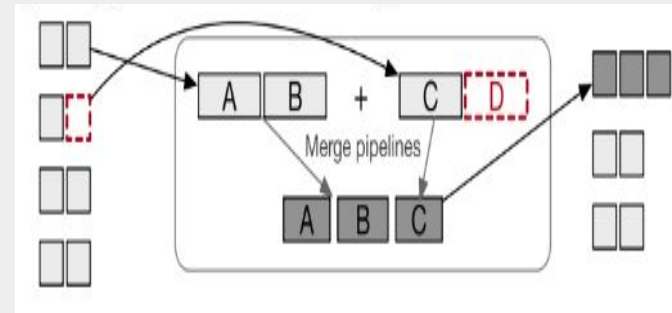
Recovery Mechanism

- IF there EXISTS a template that can accommodate remaining nodes, instantiate using the template
- IF there ISN'T a template that can accommodate remaining node,
 - If there exists a node that can be borrowed from other pipeline and reinstantiate two pipelines
 - If nodes can't be borrowed, merge with other pipeline and reinstantiate.



Recovery Mechanism

- IF there EXISTS a template that can accommodate remaining nodes, instantiate using the template
- **IF there ISN'T a template that can accommodate remaining node,**
 - If there exists a node that can be borrowed from other pipeline and reinstantiate two pipelines
 - **If nodes can't be borrowed, merge with other pipeline and reinstantiate.**



Oobleck Evaluation

Overview

Varuna : A resilient training framework based on automated parallel configuration and checkpoints

Bamboo : A resilient training framework based on redundant computation without full restart.

Oobleck used 30 NVIDIA A40 GPUs with 40GB GPU memory each. The GPUs are connected to each other via a 200Gbps Mellanox ConnectX-6 InfiniBand adaptor for communication.

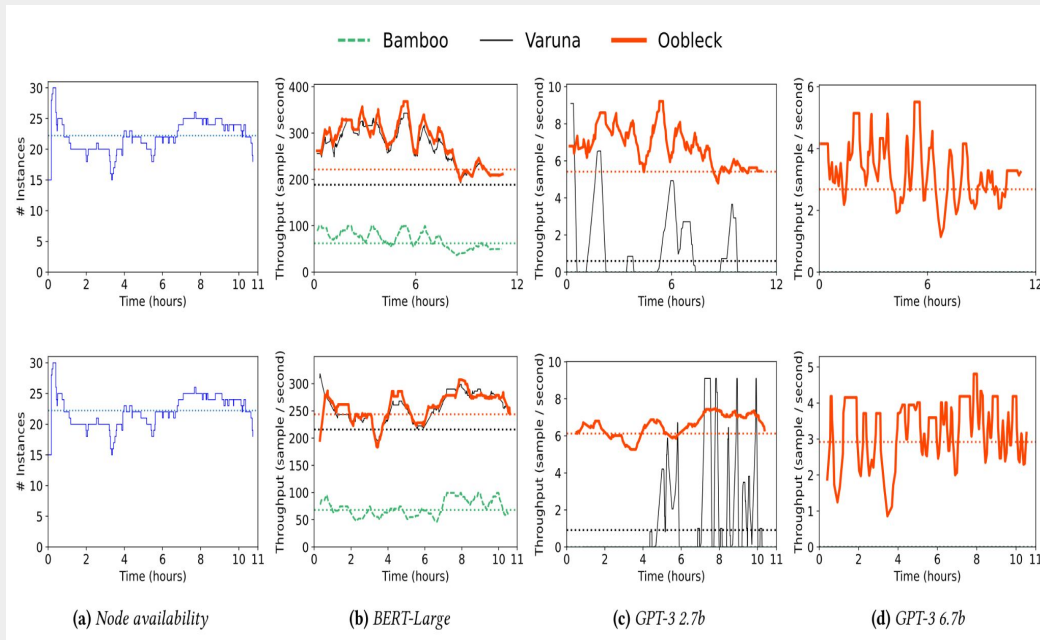
Throughput Comparison in real-world environment

Method : Simulates node failures following historical availability traces.

Bamboo : Relatively stable throughput
Overall low throughput.
Out of memory issue on large model

Varuna : Big checkpoint overhead in large model
Progress halts in high failure frequency

Oobleck : Best performance in every scenario



Overhead Analysis

Bamboo : Too much overhead due to redundant computation

Varuna : Big overhead in loading from checkpoint. Bigger fallbacks due to the starting from latest checkpoint

Oobleck : Best performance in every scenario. Lose one iteration instead of losing all computation after the checkpoint

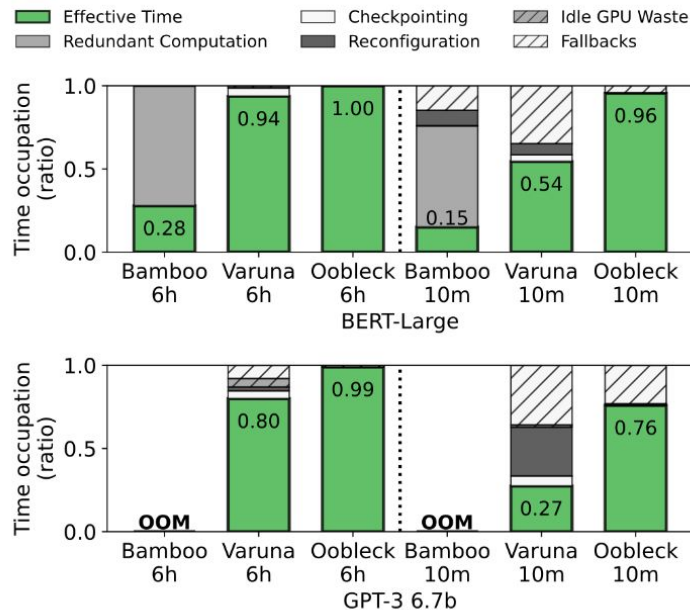


Figure 11. Time occupation breakdown of Bamboo, Varuna, and Oobleck running BERT-Large and GPT-3 6.7b model.

SuperBench (Optional Reading)

Background - **Gray Failure**

Cloud AI training relies on hardware redundancy

- Thousands of GPUs for weeks/months
- Multiple components to prevent complete failures

Gray failures: subtle performance degradation

- Components partially functional but underperforming
- Silently waste resources
- Hard to detect

Motivation

- Current approaches have limitations:
 - **Fault-tolerant systems (e.g., Oobleck)** handle crashes but may mask degraded nodes
 - **Verification systems (e.g., TrainVerify)** ensure correctness, not performance
- **Need:** Proactive validation to detect gray failures

SuperBench

Selector : Selects nodes for benchmark based on

- How long since it has been tested
- Incident Probability
- Validation time vs issue-related penalty

Validator : Runs a benchmark on the selected node

- If the benchmark result has low similarity with criteria, remove the node

Node 1 # incident : 3	Node 2 # incident : 1	Node 3 # incident: 0
Node 4 # incident : 2	Node 5 # incident : 4	Node 6 # incident : 3

SuperBench

Selector : Selects nodes for benchmark based on

- How long it has been tested
- Incident Probability
- Validation time vs issue-related penalty

Validator : Runs a benchmark on the selected node

- If the benchmark result has low similarity with criteria, remove the node

Node 1 # incident : 3	Node 2 # incident : 1	Node 3 # incident: 0
Node 4 # incident : 2	Node 5 # incident : 4	Node 6 # incident : 3

SuperBench

Selector : Selects nodes for benchmark based on

- How long it has been tested
- Incident Probability
- Validation time vs issue-related penalty

Validator : Runs a benchmark on the selected node

- If the benchmark result does not meet criteria, remove the node

Node 1 # incident : 3	Node 2 # incident : 1	Node 3 # incident: 0
Node 4 # incident : 2	Node 5 # incident : 4	Node 6 # incident : 3

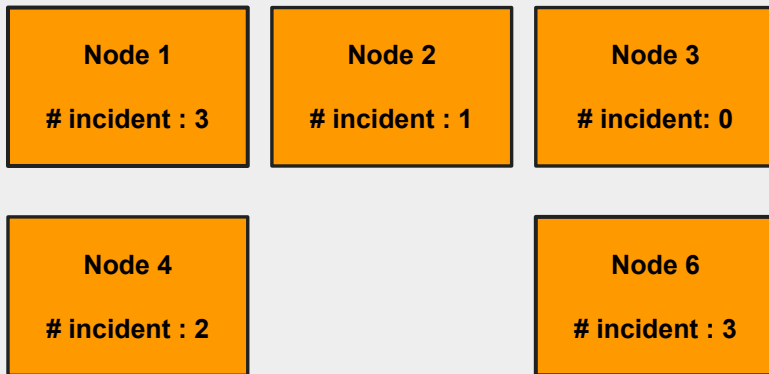
SuperBench

Selector : Selects nodes for benchmark based on

- How long it has been tested
- Incident Probability
- Validation time vs issue-related penalty

Validator : Runs a benchmark on the selected node

- If the benchmark result does not meet criteria, remove the node



Conclusion + Thoughts

Differences

Approach

- **TrainVerify**: proactive verification
- **Oobleck**: reactive recovery

Logical Equivalence

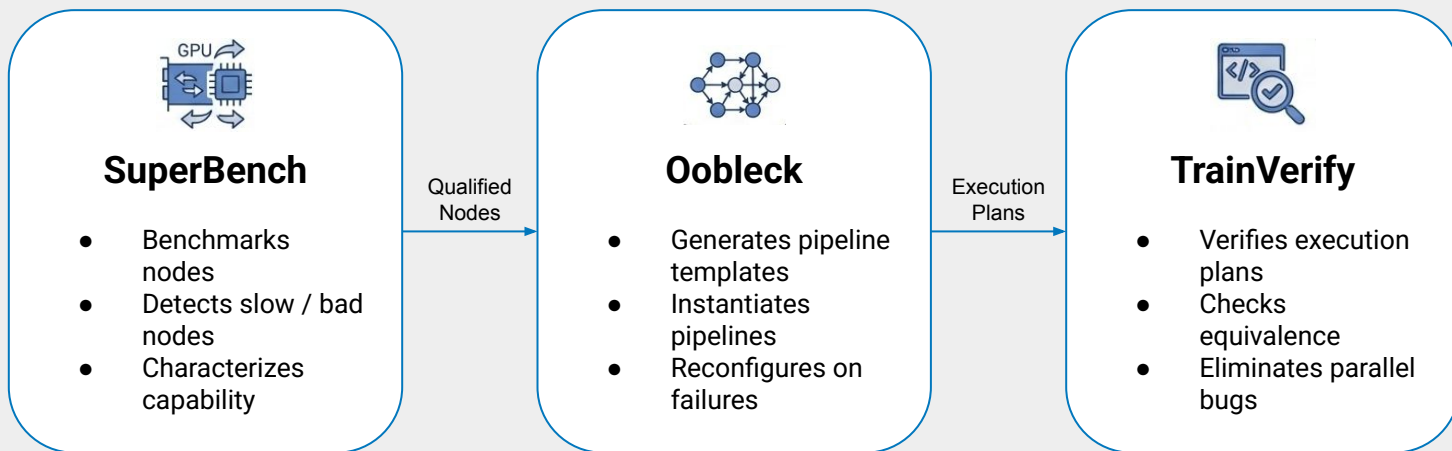
- **TrainVerify**: explicitly verifies execution graphs
- **Oobleck**: assumes equivalence by construction

Redundancy Tradeoffs

- **Oobleck**: improves reliability
- **SuperBench**: can obscure performance degradation (aka “gray-failure”)



Integration Plan





TrainVerify

Strengths

- ✓ Works with full execution plans (no homogeneity assumptions)
 - Oobleck
- ✓ Eliminates parallelization bugs that look like numeric issues
 - e.g., missing AllReduce
- ✓ System-agnostic interface
 - SSA graph + lineage

Weaknesses

- ✗ Narrow focus on parallelization issues
- ✗ Not **directly** applicable to code-based parallelization (like MegatronLM)
 - no explicit graphs or lineage
- ✗ Requires extensions for systems like XLA auto-sharding
 - lineage lost during optimization

TrainVerify excels at semantic correctness of distributed execution, but requires explicit graph structure.



Oobleck

Strengths

- ✓ Pipeline template abstraction
- ✓ Guaranteed fault-tolerance with no steady-state overhead
- ✓ Designed for real-world cluster behavior

Weaknesses

- ✗ Assumes correctness by construction
- ✗ Assumes homogeneous nodes during planning
 - same GPU type, compute capacity, memory, and network

Oobleck prioritizes efficient, fault-tolerant execution under failures, but assumes correctness and homogeneous resources.

Directions of Future Research

Automated Symbolic Reasoning (TrainVerify)

- Automatic shape reduction
- Operator rewrites for symbolic tensors
- Leveraging autograd and program synthesis

Heterogeneity-Aware Execution (Oobleck)

- Performance under heterogeneous hardware
- Capability-aware pipeline planning

Q & A